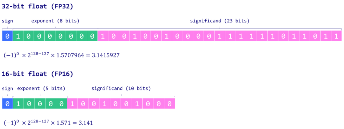
# Deep Learning for NLP

# Training LLMs
# Memory and compute requirements




## Learning goals

- Learn about different contributions to compute requirements
- Learn how model size components influence memory requirements

# NUMBER OF PARAMETERS: NOTATION

- In this slide set, we use $P$ for the number of parameters.
- (Unfortunately, we use $N$ for the number of parameters in other slide sets.)

# COMPUTE REQUIREMENTS

# COMPUTE REQUIREMENTS

**Basic equation: Cost to train a transformer (decoder) model:**

$$C \approx \tau T = 6PD$$

Source: Quentin et al., 2023

# COMPUTE REQUIREMENTS $C \approx \tau T = 6PD$

**where:**

- $\tau$ is throughput of hardware: (No. GPUs) x (FLOPs/GPU)
- $T$ is the time spent training the model, in seconds
- $P$ is the number of parameters in the model
- $D$ is the dataset size (in tokens)
- $C$: No. of floating-point operations to train the model:
  $C = C_{forward} + C_{backward}$
- $C_{forward} \approx 2PD$; $\quad C_{backward} \approx 4PD$
  - $2PD$: **2** comes from the multiply-accumulate operation used in matrix multiplication
  - $4PD$: backward pass is approximately twice the compute of the forward pass
  - In the backward pass at each layer, gradients have to be calculated for the weights at that layer and for the previous layers' outputs.

# COMPUTE UNITS

$C$ (actually, $C$ per time) can be measured in different units:

- FLOPs = FLOP-seconds which is [Floating Point Ops / Second]
  - We also use multiples:
    GFLOP-seconds, TFLOP-seconds etc.
  - Other multiples like PFLOP-days are used in papers
  - 1 PFLOP-day = $10^{15} \cdot 24 \cdot 3600$ FLOP-seconds
  - Actual FLOPs are always lower than the advertised theoretical FLOPs
- GPU-hours
  - GPU model is also required, since they have different compute capacities

# PARAMETERS VS DATASET

- Model performance depends on number of parameters $P$, but also on number of training tokens $D$
- One proposed optimal tradeoff between $P$ and $D$ is: $D = 20P$
  - This was posited for Chinchilla models ▸ Hoffmann et al., 2022, no longer seen as valid.
  - See next lecture
- You need a training set of several 100s of billions of tokens at least (e.g., LLM does not become fluent otherwise).
- Two typical ways of determining $P$ (the size of the model) in 2026:
  - Based on available compute budget: how much data and how many parameters are optimal?
  - Based on a desired inference time performance (e.g., generate 100 tokens per second): how much data and how many parameters are optimal?

# MEMORY REQUIREMENTS

# MEMORY REQUIREMENTS

Common questions:

- How big is this model in bytes?
- Will it fit/train on my GPUs?

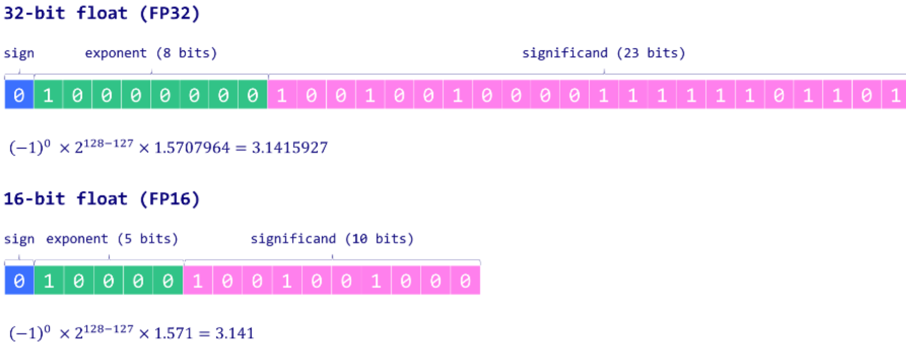
Model size components:

- Model parameters
- Optimizer states
- Gradients
- Activations

# NUMBER REPRESENTATIONS

- fp32: single precision floating point number as defined by ▸ IEEE 754 standard, takes 32 bits or 4 bytes
- fp16: half precision float number as defined by ▸ IEEE 754-2008, occupying 16 bits or 2 bytes
- bf16 or brain floating point 16, developed by Google Brain project, occupying 16 bits or 2 bytes
  - fewer bits for the "significand" than fp16 $\rightarrow$ bf16 is less precise
  - larger dynamic range than fp16 (8 exponent bits instead of 5) $\rightarrow$ less likely to suffer from underflow or overflow
- INT8: integer (e.g., from -128 to 127), occupying 8 bits or 1 byte

# FP32/FP16 ((MAXIME LABONNE))



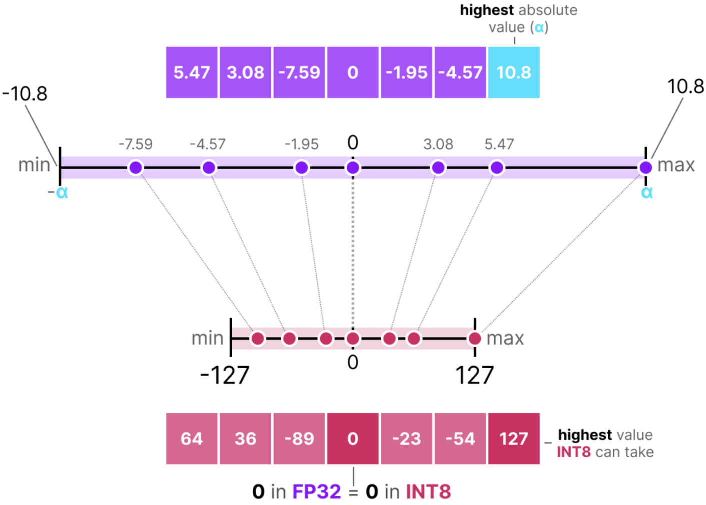

You can represent every float like this: $(-1)^{S} \cdot 2^{e-bias} \cdot 1.m$

# INT8 QUANTIZATION

*It's harder (though not impossible) to fit meaningful floats into 8 bits.*
*Alternative: use integer quantization: INT8.*

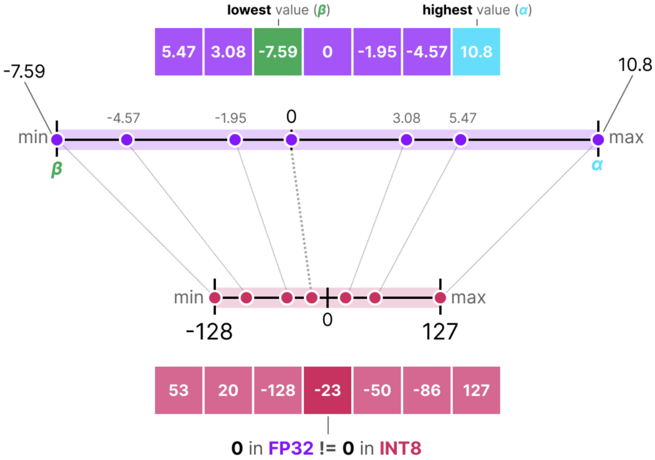
- Real_number = stored_integer * scaling_factor
- **Absmax quantization (symmetric)**:
  - $X_{quant} = \text{round}\left(\frac{127}{max|\mathbf{X}|} \cdot \mathbf{X}\right)$
- **Zero-point quantization (assymmetric)**:
  - $\text{scale} = \frac{255}{max(\mathbf{X}) - min(\mathbf{X})}$
  - $\text{zeropoint} = -round(\text{scale} \cdot min(\mathbf{X})) - 128$
  - $X_{quant} = round(\text{scale} \cdot \mathbf{X} + \text{zeropoint})$

# ABSMAX INT8 (SYMMETRIC)




Link: Maarten Grootendorst

# ZERO-POINT INT8 (ASYMMETRIC)




Link: Maarten Grootendorst

# ZERO-POINT QUANTIZATION

- We assume: $\mathit{min}(\mathbf{X}) \leq 0 \leq \mathit{max}(\mathbf{X})$
- scale $= \frac{255}{\mathit{max}(\mathbf{X}) - \mathit{min}(\mathbf{X})}$
- zeropoint $= -\mathit{round}(\text{scale} \cdot \mathit{min}(\mathbf{X})) - 128$
- $X_{quant} = \mathit{round}(\text{scale} \cdot \mathbf{X} + \text{zeropoint})$
- $X_{quant}(0) = \mathit{zeropoint}$
- $X_{quant} = \mathit{round}(\text{scale} \cdot \mathbf{X} + \text{zeropoint})$
  $= \mathit{round}(\text{scale} \cdot \mathbf{X} + -\mathit{round}(\text{scale} \cdot \mathit{min}(\mathbf{X}) - 128))$
  $\approx \mathit{round}(\text{scale} \cdot (\mathbf{X} - \mathit{min}(\mathbf{X}))) - 128$
  $= \mathit{round}(255 \cdot \frac{\mathbf{X} - \mathit{min}(\mathbf{X})}{\mathit{max}(\mathbf{X}) - \mathit{min}(\mathbf{X})}) - 128$
- $X_{quant}(\mathit{max}(\mathbf{X})) = 255 - 128 = 127$
- $X_{quant}(\mathit{min}(\mathbf{X})) = 0 - 128 = -128$

# ACTIVE AREA OF RESEARCH

NVIDIA

2026-3-6

## Pretraining Large Language Models with NVFP4

**NVIDIA**

**Abstract.** Large Language Models (LLMs) today are powerful problem solvers across many domains, and they continue to get stronger as they scale in model size, training set size, and training set quality, as shown by extensive research and experimentation across the industry. Training a frontier model today requires on the order of tens to hundreds of yottaflops, which is a massive investment of time, compute, and energy. Improving pretraining efficiency is therefore essential to enable the next generation of even more capable LLMs. While 8-bit floating point (FP8) training is now widely adopted, transitioning to even narrower precision, such as 4-bit floating point (FP4), could unlock additional improvements in computational speed and resource utilization. However, quantization at this level poses challenges to training stability, convergence, and implementation, notably for large-scale models trained on long token horizons.

Link: Nvidia paper on NVFP4

# MODEL PARAMETERS

Parameter size depends on chosen representation:

- fp32: $Mem_{model} = 4 \ bytes/param \cdot N_{params}$
- fp16 or bf16: $Mem_{model} = 2 \ bytes/param \cdot N_{params}$
- INT8: $Mem_{model} = 1 \ byte/param \cdot N_{params}$

It is common to use mixed representations:

- fp32 + fp16
- fp32 + bf16

# OPTIMIZER STATES

AdamW: $Mem_{\text{AdamW}} = 8\ bytes/param \cdot N_{params}$

- Momentum: 4 bytes/param
- Variance: 4 bytes/param

bitsandbytes (8-bit optimizer): $Mem_{\text{optimizer}} = 2\ bytes/param \cdot N_{params}$

- Momentum: 1 byte/param
- Variance: 1 byte/param

# GRADIENTS

They are usually stored in the same datatype as the model parameters.

Their memory overhead contribution is:

- fp32: $Mem_{grad} = 4\ bytes/param \cdot N_{params}$
- fp16 or bf16: $Mem_{grad} = 2\ bytes/param \cdot N_{params}$
- INT8: $Mem_{grad} = 1\ byte/param \cdot N_{params}$

# ACTIVATIONS

- GPUs are bottlenecked by memory, not FLOPs
- Save GPU memory by recomputing activations of certain layers
- Various schemes for how exactly this is implemented.

Total memory when training **without** activations:
$Mem_{training} = Mem_{params} + Mem_{opt} + Mem_{grad}$

Total memory when training **with** activations:
$Mem_{training} = Mem_{params} + Mem_{opt} + Mem_{grad} + Mem_{activ}$